

LECTURE 9

Multicore Programming and Multithreading Models

CSC323 · Operating Systems

COMSATS University Islamabad, Lahore Campus

In This Lecture

- Five concrete challenges multicore hardware introduces for programmers, beyond single-core concurrency
- User threads vs. kernel threads, and why the relationship between them matters
- The three multithreading models — many-to-one, one-to-one, and many-to-many — and their tradeoffs
- Where Pthreads, Windows threads, and Java threads fit into that picture

From Interleaving to True Parallelism

A single-core system running multiple threads only ever interleaves them — one core switching between threads so fast it looks simultaneous. A multicore system breaks that assumption: with N cores, up to N threads can execute truly at the same instant.

Single Core

Threads are interleaved. At any instant, exactly one instruction from exactly one thread executes. Simultaneity is an illusion created by fast switching.

Multicore

Threads genuinely run at the same instant. With N cores, up to N threads make independent forward progress at the same moment in real time.

More cores only pay off if the programmer's code is structured to take advantage of them — restructuring sequential logic into concurrent logic raises five distinct problems.

Five Multicore Programming Challenges

Identifying tasks

Finding pieces of the program that can genuinely run independently

Balance

Giving each core roughly equal-value work, so none sits idle

Data splitting

Dividing the data a task operates on into safely divisible pieces

Data dependency

One task's output feeds another's input — a hard limit on parallel speedup

Testing and debugging

Genuinely simultaneous execution multiplies possible interleavings

Identifying Tasks & Balance

Identifying Tasks

- **Architectural, not algorithmic:** find pieces of work that don't depend on one another's results
- **Good split:** resizing 10,000 independent image files — image #342 has nothing to do with image #7,891
- **Poor split:** a running total read line-by-line, where each step needs the previous step's result

Balance

- **Not enough to find independent tasks** — they must be close to equal in size
- **Example:** one 10s task + three 1s tasks on four cores leaves three cores idle for 9 seconds
- **Balance means dividing the value** of the work evenly, not just the count of tasks

Data Splitting & Data Dependency

Data Splitting

- **The data a task operates on** must be divided safely alongside the tasks themselves
- **Safe:** give each core a contiguous quarter of a large array — no two cores touch the same elements
- **Unsafe:** give every core a pointer to the whole array and trust them not to collide

Data Dependency

- **One task's output is another task's input** — step B cannot start until step A finishes
- **The two steps are serialized**, however many idle cores are sitting around
- **Dependency points set a hard ceiling** on the speedup more cores can ever deliver

Testing and Debugging

- **Single-core concurrency bugs are already hard to reproduce** — they depend on exactly when a context switch lands
- **True parallelism multiplies this:** with threads genuinely running at the same instant on different cores, the number of possible relative orderings explodes
- **A bug tied to one rare ordering** might show up once in ten thousand runs on a developer's machine, then corrupt production data
- **This motivates the formal synchronization tools** the next unit introduces — “just be careful” does not scale to this many interleavings

More cores are not a free performance upgrade

A program written as one long sequential chain of dependent steps runs no faster on a 16-core machine than on a 1-core machine. Identifying tasks, balancing them, and respecting data dependencies is work the programmer has to do — the hardware cannot discover parallelism that was never designed into the program.

User Threads vs. Kernel Threads

Who actually schedules threads onto the CPU? The answer splits threads into two kinds that live at two different levels — and the mapping between them is exactly what a multithreading model defines.

User Thread

Managed entirely by a thread library running in user space (e.g. Pthreads). The kernel doesn't necessarily know these threads exist as separate entities at all.

Kernel Thread

Managed directly by the operating system kernel — the only thing that can actually schedule work onto a physical core.

Because user threads are what the programmer creates and kernel threads are what the hardware actually runs, there must be a mapping between the two. There are three such models.

Many-to-One

Many user threads map onto a single kernel thread. The thread library does all scheduling of user threads in user space; the kernel sees the whole process as one schedulable entity.

- **Advantage:** thread creation and context switching stay entirely in user space, with no kernel involvement — both are very fast
- **Disadvantage:** if one user thread makes a blocking system call, the single kernel thread blocks — the entire process stops, even threads with nothing to do with the call
- **Disadvantage:** only ever one kernel thread, so the kernel can schedule at most one of this process's threads at a time — no true parallelism, even with many idle cores

One-to-One

Each user thread maps to its own, dedicated kernel thread.

- **Advantage:** true parallelism — the kernel can schedule several of this process's threads onto several cores at the same instant; one thread blocking on I/O never affects another
- **Disadvantage:** every thread creation requires creating a corresponding kernel thread, which is comparatively expensive, and many implementations cap how many a process may create

This is what Linux and Windows actually use

Despite the overhead, the one-to-one model's simplicity and guaranteed parallelism won out in practice — both Linux (via NPTL) and Windows map every user thread directly onto its own kernel thread.

Many-to-Many

Many user threads are multiplexed onto a smaller or equal number of kernel threads — not forced down to one, and not forced up to one-per-thread either.

- **Advantage:** create as many user threads as the problem calls for without one-to-one's full kernel-thread cost, while the kernel still runs several threads in parallel
- **Advantage:** if one user thread blocks, the thread library can move a different ready user thread onto the now-free kernel thread — the process doesn't stall
- **Disadvantage:** real implementation complexity — the thread library and kernel scheduler must cooperate (e.g. Solaris scheduler activations) to decide which user thread rides which kernel thread

The Three Multithreading Models, Compared

Many-to-One

Many user threads → 1 kernel thread. Cheapest to create/switch; one blocking call stalls everything; zero parallelism on multicore.

One-to-One

1 user thread → 1 kernel thread each. True parallelism; costlier creation; often a cap on thread count. Used by Linux and Windows.

Many-to-Many

Many user threads → fewer-or-equal kernel threads. Best of both — but the most complex model to implement correctly.

The question to ask: “if one thread makes a blocking system call, what else stops?” and “if I have 8 idle cores, can this process actually use more than one at once?”

Model Comparison, Side by Side

	Many-to-One	One-to-One	Many-to-Many
Creation / switch cost	Very low (user space only)	Higher (kernel involved every time)	Low-to-moderate
Parallelism on multicore	None	Full	Full (bounded by kernel threads)
One thread blocks on I/O	Entire process blocks	Only that thread blocks	Library can migrate another thread in
Implementation complexity	Simple	Simple	Hard — needs library/kernel cooperation

Multithreading Libraries

A multithreading library is what a programmer actually calls to create and manage threads — and which model backs that library depends on the operating system underneath it.

- **Pthreads** — the POSIX standard for thread creation and synchronization (pthread_create, pthread_join, mutexes/condition variables). Depending on the OS, Pthreads can sit on any of the three models — on Linux, it's one-to-one.
- **Windows threads** — kernel-level by design, matching Windows' own one-to-one model directly; there is no separate user-level Windows thread.
- **Java threads** — managed by the JVM, but not an independent fourth model — implemented using whatever threading model the host OS provides (Pthreads on Linux, native threads on Windows).

A Minimal Pthreads Example

```
#include <pthread.h>
#include <stdio.h>

void *print_hello(void *arg) {
    long id = (long) arg;
    printf("Hello from thread %ld\n", id);
    return NULL;
}

int main(void) {
    pthread_t threads[4];

    for (long i = 0; i < 4; i++)
        pthread_create(&threads[i], NULL, print_hello, (void *) i);

    for (int i = 0; i < 4; i++)
        pthread_join(threads[i], NULL);

    return 0;
}
```

Reading the Example

- **pthread_create()** spawns a new thread running `print_hello`, passing it one argument — called once per thread, four times in the loop
- **pthread_join()** blocks the calling thread until the specified thread finishes
- **Without pthread_join()**, `main()` could reach `return 0` and tear the process down before the four threads ever got to print anything
- **This create-then-join shape** is the one every C thread program shares: spawn work, then wait for it to finish

Key Takeaways

- Multicore hardware turns threads from an interleaving illusion into genuine simultaneous execution — which only pays off if tasks are identified, balanced, and their data dependencies respected
- A user thread is managed by a thread library in user space; a kernel thread is scheduled directly by the OS — the multithreading model is the mapping between them
- Many-to-one is cheap but blocks entirely on one thread's call and gets no multicore parallelism; one-to-one (Linux, Windows) gets full parallelism at a higher creation cost; many-to-many tries for both, at the price of real complexity
- Pthreads, Windows threads, and Java threads are libraries, not models — each inherits its scheduling behavior from whichever model the underlying OS implements
- Testing and debugging genuinely parallel code is fundamentally harder, which motivates the formal synchronization tools covered starting in Lecture 13